Capturing TCP and TLS Handshake Using Network Namespaces

Samyak Gedam (252IS032)

February 2026

Aim:
To create two network namespaces (red and blue), establish a secure HTTPS connection between them, capture the TLS 3-way handshake, and analyze the captured packets in Wireshark.

Tools Used:
Linux Network Namespaces, OpenSSL, curl, tcpdump, Wireshark

Description:
Two network namespaces named **red** and **blue** were created and connected using virtual interfaces. The blue namespace acted as server. The red namespace acted as the client and initiated an HTTPS request using the curl command. During this communication, tcpdump was used in the blue namespace to capture the packetse and save it into a `.pcap` file. The captured file was later analyzed using Wireshark.

Commands Used

Certificate Generation:

```
openssl genrsa -out server.key 2048
openssl req -new -x509 -key server.key -out certificate.crt -days 365
```

Starting TLS Server in Blue Namespace:

```
sudo ip netns exec blue openssl s_server -accept 443 \
-cert certificate.crt -key server.key
```

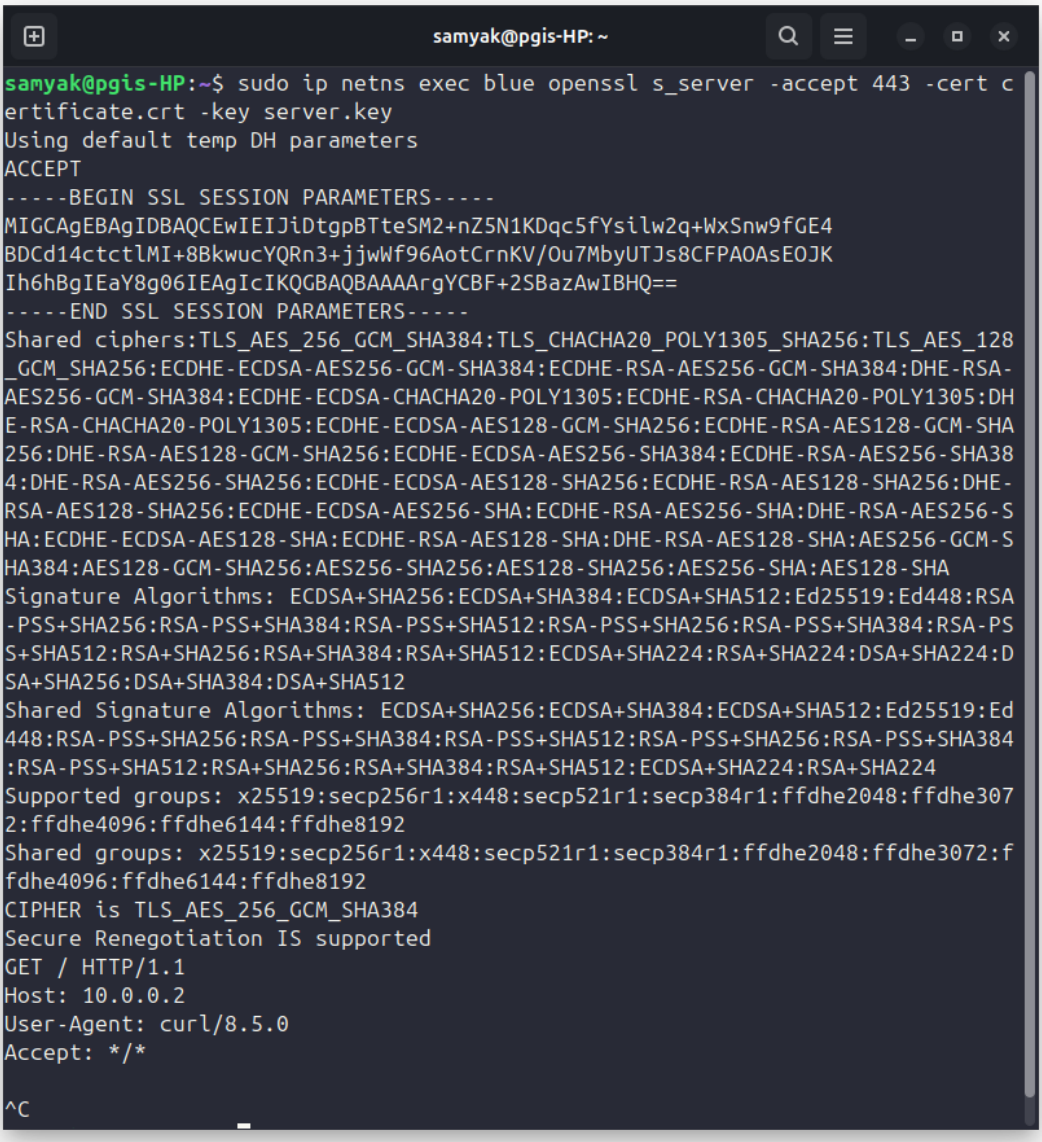
Capturing Packets:

```
sudo ip netns exec blue tcpdump -i eth1 -w tls_capture.pcap
```

Generating HTTPS Request from Red Namespace:

```
sudo ip netns exec red curl -k https://10.0.0.2
```

Screenshots:



```
samyak@pgis-HP: ~
samyak@pgis-HP:~$ sudo ip netns exec blue openssl s_server -accept 443 -cert c
ertificate.crt -key server.key
Using default temp DH parameters
ACCEPT
-----BEGIN SSL SESSION PARAMETERS-----
MIGCAgEBAgIDBAQCEwIEIJiDtgpBTteSM2+nZ5N1KDqc5fYsilw2q+WxSnw9fGE4
BDCd14ctctlMI+8BkwucYQRn3+jjwWf96AotCrnKV/Ou7MbyUTJs8CFPAOAsEOJK
Ih6hBgIEaY8g06IEAgIcIKQGBAQBAAAArgYCBF+2SBazAwIBHQ==
-----END SSL SESSION PARAMETERS-----
Shared ciphers:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128
_GCM_SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-
AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:DH
E-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA
256:DHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES256-SHA38
4:DHE-RSA-AES256-SHA256:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-SHA256:DHE-
RSA-AES128-SHA256:ECDHE-ECDSA-AES256-SHA:ECDHE-RSA-AES256-SHA:DHE-RSA-AES256-S
HA:ECDHE-ECDSA-AES128-SHA:ECDHE-RSA-AES128-SHA:DHE-RSA-AES128-SHA:AES256-GCM-S
HA384:AES128-GCM-SHA256:AES256-SHA256:AES128-SHA256:AES256-SHA:AES128-SHA
Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA
-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PS
S+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224:DSA+SHA224:D
SA+SHA256:DSA+SHA384:DSA+SHA512
Shared Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed
448:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384
:RSA-PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224
Supported groups: x25519:secp256r1:x448:secp521r1:secp384r1:ffdhe2048:ffdhe307
2:ffdhe4096:ffdhe6144:ffdhe8192
Shared groups: x25519:secp256r1:x448:secp521r1:secp384r1:ffdhe2048:ffdhe3072:f
fdhe4096:ffdhe6144:ffdhe8192
CIPHER is TLS_AES_256_GCM_SHA384
Secure Renegotiation IS supported
GET / HTTP/1.1
Host: 10.0.0.2
User-Agent: curl/8.5.0
Accept: */*

^C
```

Figure 1: Captured traffic at server and stored in a pcap file

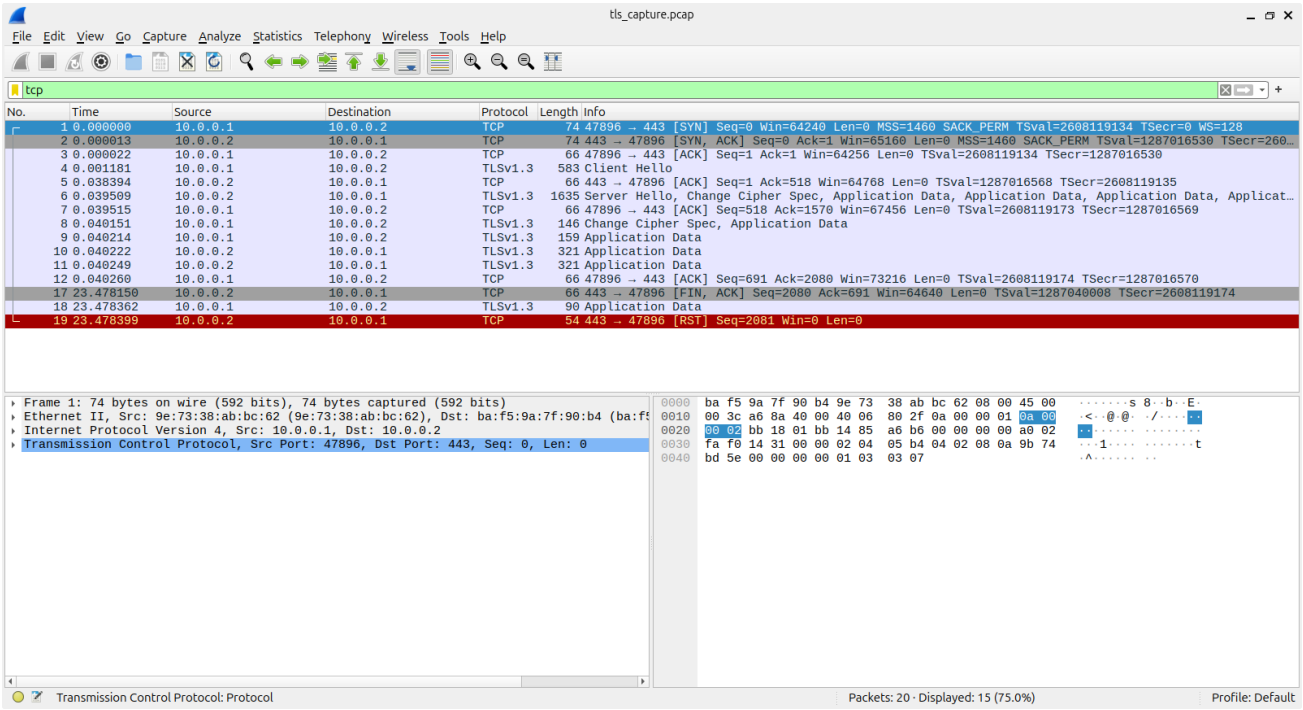


Figure 2: Opening pcap in Wireshark

This is a Wireshark capture of a TLS 1.3 connection between:

- Client: 10.0.0.1
- Server: 10.0.0.2
- Service Port: 443 (HTTPS)

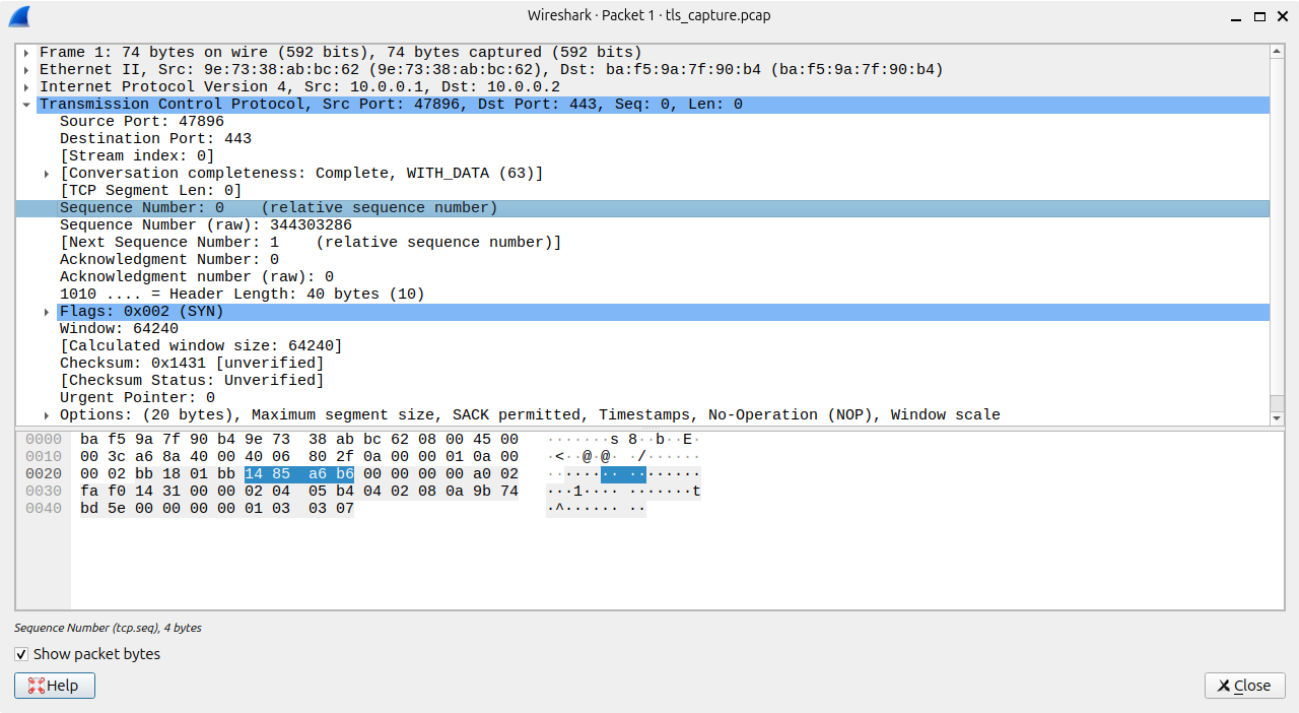


Figure 3: TCP Initial SYN

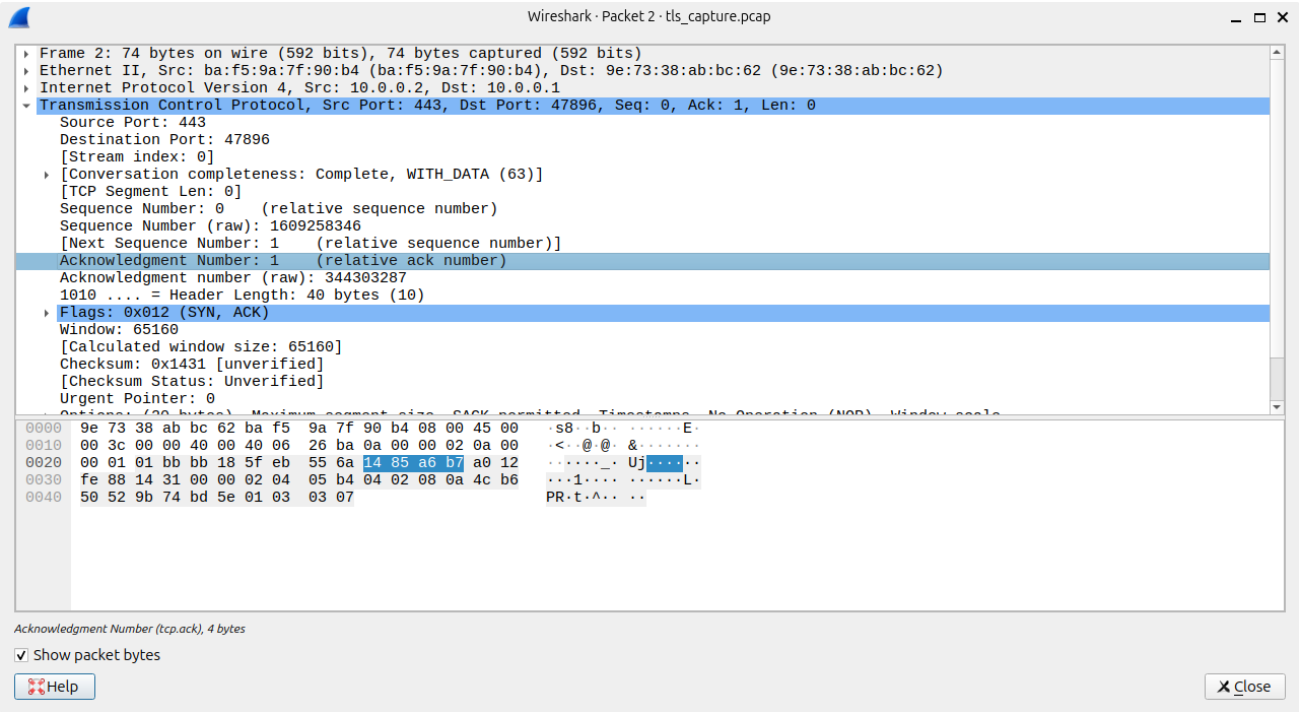


Figure 4: TCP SYN-ACK

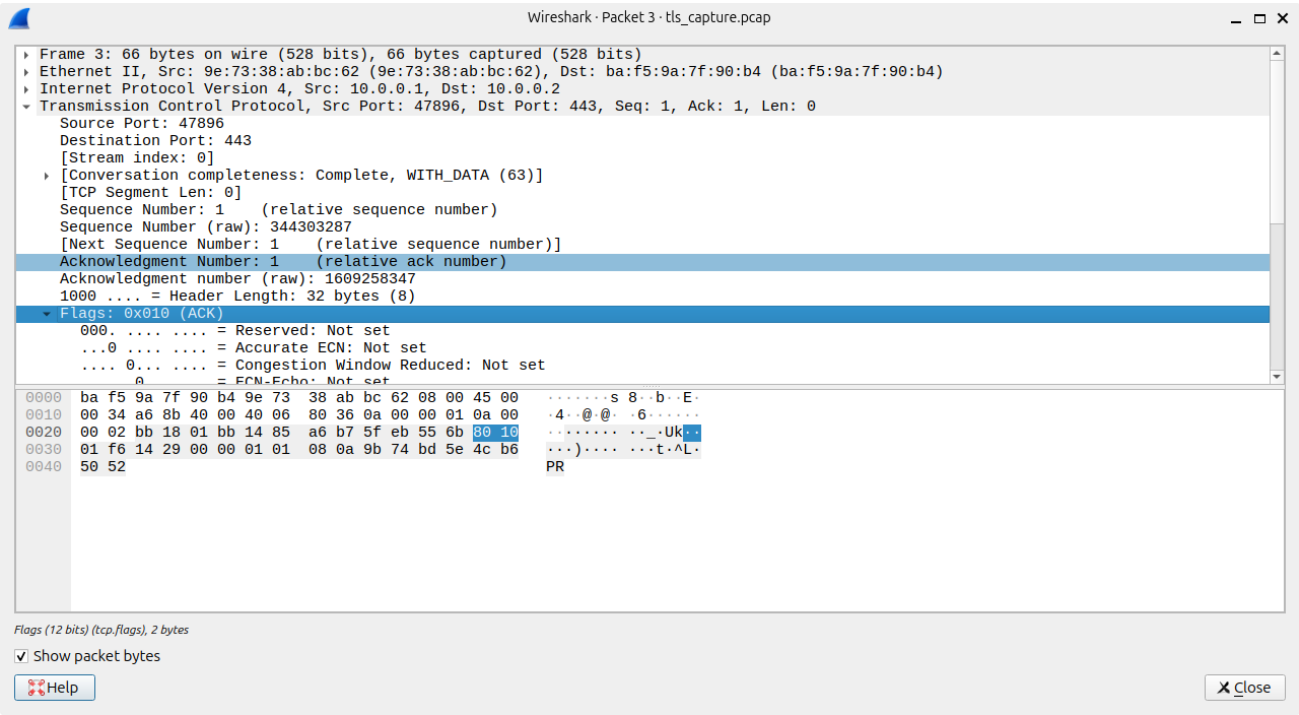


Figure 5: TCP Final ACK

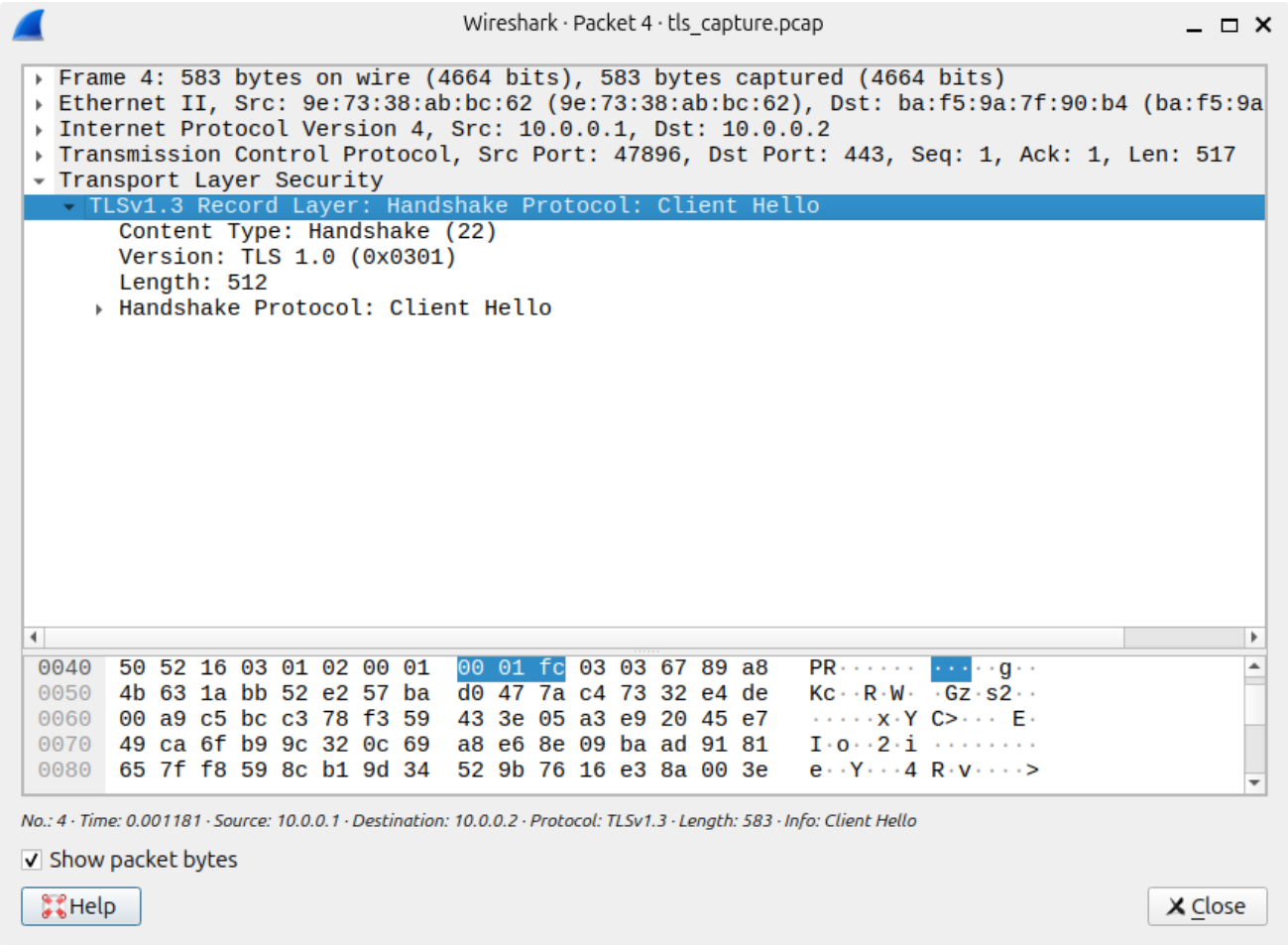


Figure 6: TLSv1.3 Client Hello

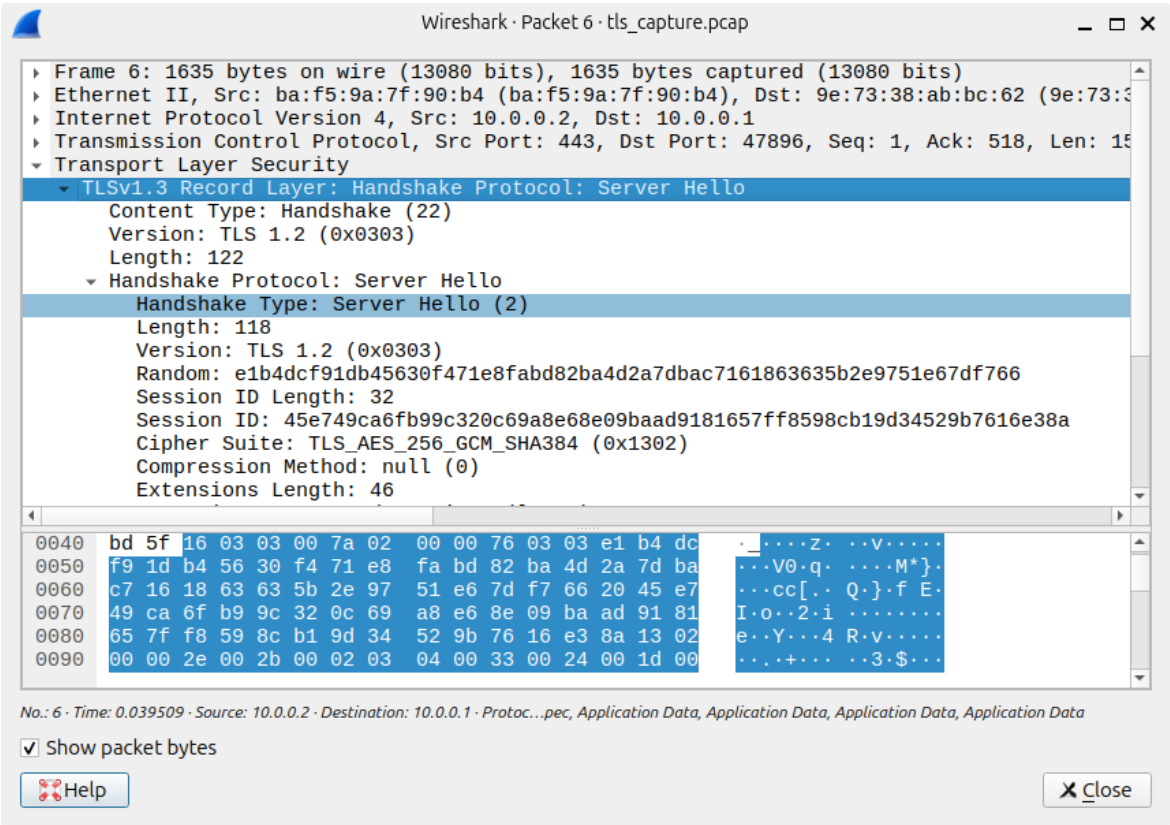


Figure 7: TLSv1.3 Server Hello

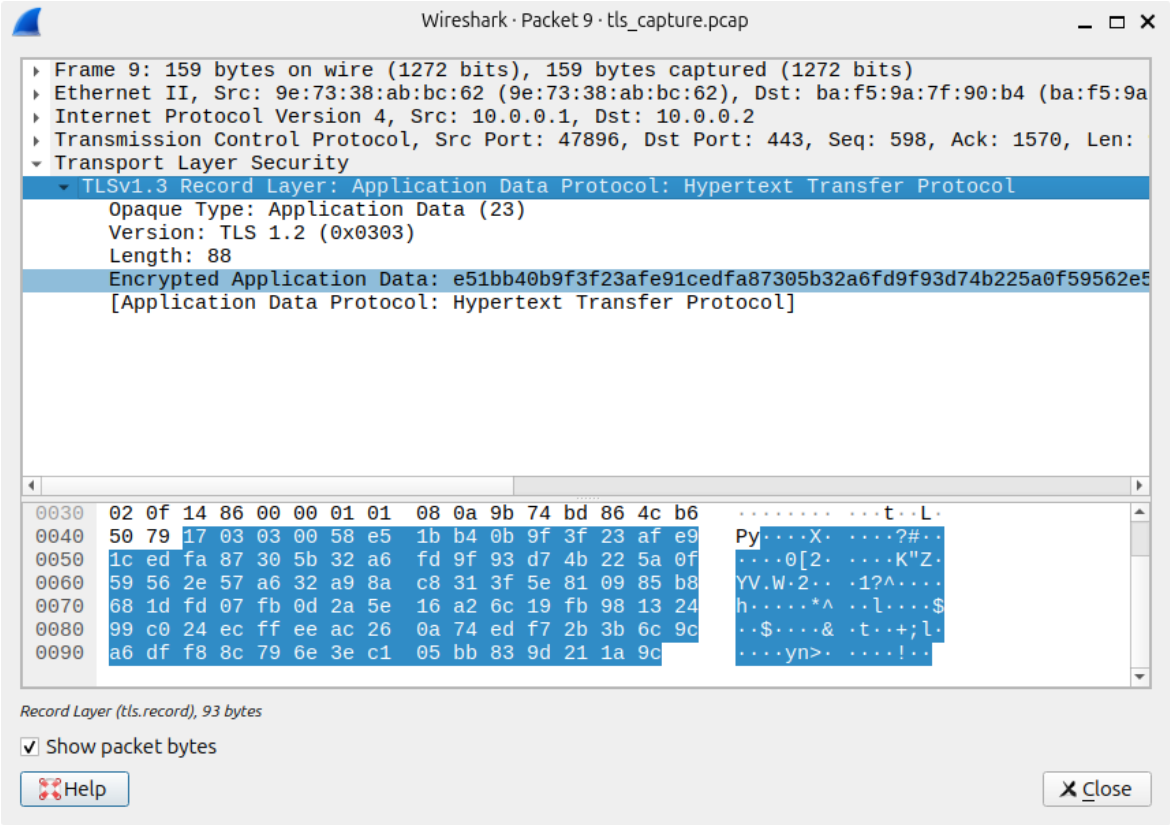


Figure 8: Encrypted Data Transfer

After Server Hello, TLS 1.3 encrypts everything.

It's actually encrypted handshake messages like: Encrypted Extensions, Certificate, Certificate Verify, Finished.

TLS 1.3 hides handshake contents.

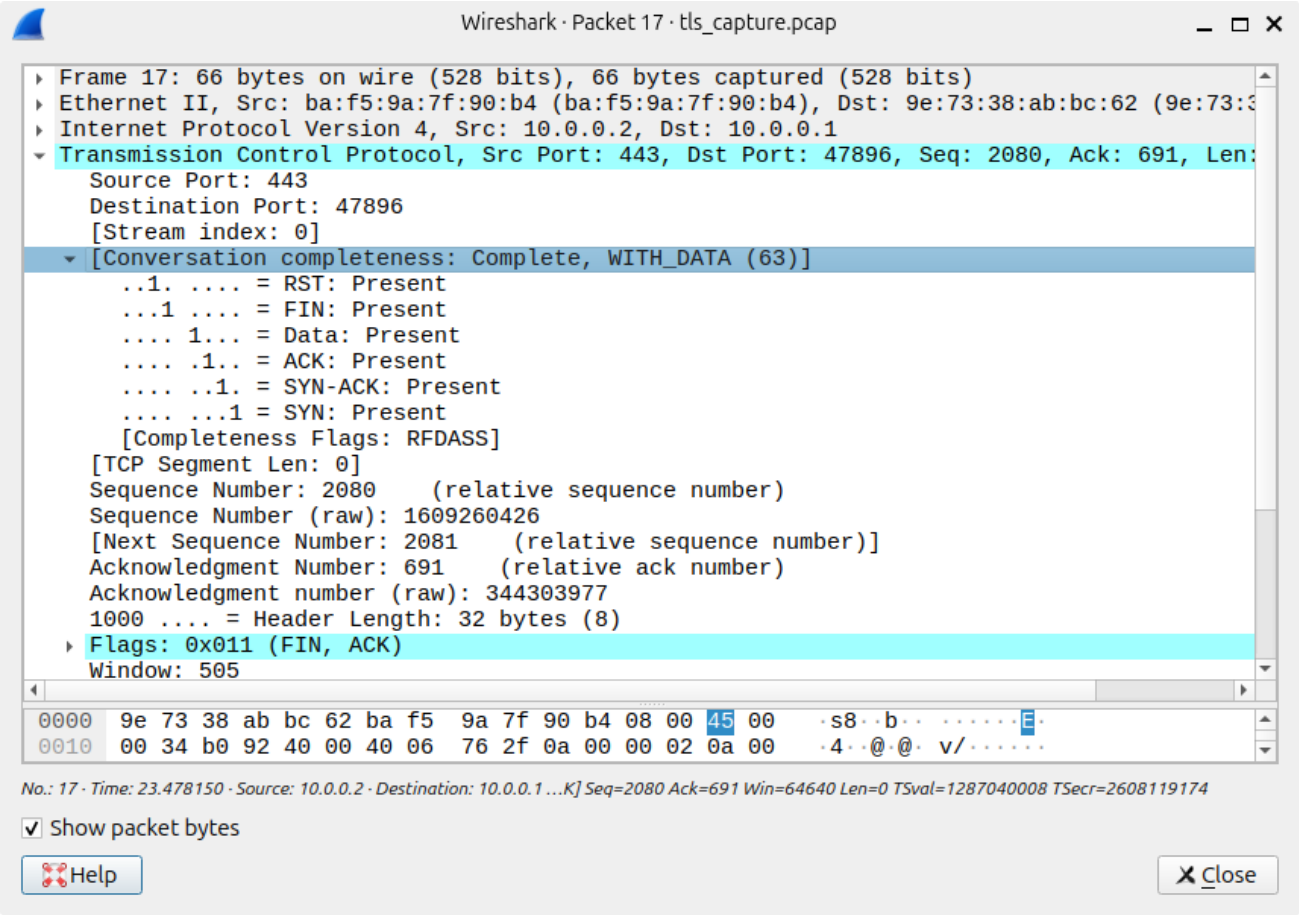


Figure 9: Connection Termination

Result

The TCP 3-way handshake and TLS 1.3 handshake between the red and blue namespaces were successfully captured and analyzed using Wireshark.